

Assignment 4: Face detection with a sliding window

概述

本次作业实现了基于滑动窗口 + HoG 特征 + 线性 SVM 的人脸（头部）检测器。主要完成如下模块：

- 从正样本提取 HoG 特征（包含镜像增强）；
- 从非人脸场景中随机采样负样本并转换为 HoG 特征；
- 利用训练好的分类器挖掘难负样本并扩充训练集；
- 训练线性 SVM (`sklearn.svm.LinearSVC`)；
- 多尺度滑动窗口检测与非极大值抑制 (NMS)。

本报告概述实现细节、关键超参、实验结果与评分估计。检测结果截图在文末。

实现细节

- 文件：`student_code.py` 中实现了以下函数：
 - `get_positive_features(train_path_pos, feature_params)`：读取 36x36 的正样本，计算 HoG，并对每张正样本做左右镜像增强。
 - `get_random_negative_features(non_face_scn_path, feature_params, num_samples)`：在非人脸场景上计算整图 HoG，然后随机取 HoG 窗口作为负样本（多图、多位置采样）。
 - `mine_hard_negs(non_face_scn_path, svm, feature_params)`：在非人脸场景上滑动窗口使用当前分类器打分，筛选出得分 > 0 的假阳性作为难负样本返回。
 - `train_classifier(features_pos, features_neg, C)`：使用 `LinearSVC(C=C, max_iter=10000, dual=False)` 训练线性分类器并返回模型。
 - `run_detector(test_scn_path, svm, feature_params, verbose=False)`：在每张测试图上按多个尺度计算 HoG，在 HoG 单元格上滑动模板大小的窗口进行分类，阈值过滤后对候选框使用 `non_max_suppression_bbox()` 进行去重，返回 `bboxes, confidences, image_ids`。

实现要点与合理性说明：

- 所有 HoG 计算均基于 `vlfeat.hog.hog()`，在图像尺度上先计算 HoG 再在 HoG 空间提取窗口，提高效率。
- 正样本做了左右镜像以扩充训练数据，有助于提高泛化。
- 负样本在 HoG 空间随机采样多个位置（每张图最多采样固定数量），支持多尺度负样本通过对不同大小图像计算 HoG 实现。

使用的超参数（代码 / notebook 中）

- 模板大小 (`template_size`) : 36
- HoG 单元格大小 (`hog_cell_size`) : 6
- 多尺度步长 (缩放因子 `scale_factor`) : 0.85
- 检测置信度阈值 (`conf_threshold`) : -2.0
- 训练负样本数 (`num_negative_examples`) : 20000
- SVM 正则项 `C`: $5e-2$ （在 notebook 中进行了尝试与调优）
- `topk/pre_nms_topk`: 500（对每张图在 NMS 前保留的最高分候选数）

这些参数在 notebook 中被记录并用于最终检测（见 [proj5.ipynb](#)）。可进一步搜索 `feature_params` 字典以调整。

实验流程与结果说明

实验流程：

1. 使用 `get_positive_features()` 构建正样本 HoG 特征集合（含镜像）。
2. 使用 `get_random_negative_features()` 在非人脸场景中随机采样负样本并构建负特征集合。
3. 用 `train_classifier()` 在正负样本上训练线性 SVM，评估训练集上的置信度分布与简单统计（见 notebook 可视化）。
4. 使用 `mine_hard_negs()` 挖掘假阳性并将其并入负样本集合，重新训练 SVM（如 notebook 所示进行了二次训练对比）。
5. 在 CMU+MIT 测试集上使用 `run_detector()` 进行多尺度检测并通过 `evaluate_detections()` 计算精确率-召回率曲线与 Average Precision（AP）。

结果截图（检测可视化）：

```
gt_ids, gt_bboxes, gt_isclaimed, tp, fp, duplicate_detections = evaluate_detections(bboxes, confidences,
                                                                                  image_ids, label_filename)

# Sort by decreasing confidence before computing AP
order = np.argsort(-confidences)
tp_sorted = tp[order]
fp_sorted = fp[order]

cum_tp = np.cumsum(tp_sorted)
cum_fp = np.cumsum(fp_sorted)
npos = len(gt_ids)
rec = cum_tp / npos
prec = cum_tp / (cum_tp + cum_fp)
ap = voc_ap(rec, prec)
print('Average Precision = {:.4f}'.format(ap))
```

Average Precision = 0.8476

```
gt_ids, gt_bboxes, gt_isclaimed, tp_2, fp_2, duplicate_detections_2 = evaluate_detections(bboxes_2, confidences_2,
                                                                                          image_ids_2, label_filename)

# Sort by decreasing confidence before computing AP
order = np.argsort(-confidences_2)
tp_sorted = tp_2[order]
fp_sorted = fp_2[order]

cum_tp = np.cumsum(tp_sorted)
cum_fp = np.cumsum(fp_sorted)
npos = len(gt_ids)
rec = cum_tp / npos
prec = cum_tp / (cum_tp + cum_fp)
ap = voc_ap(rec, prec)
print('Average Precision = {:.4f}'.format(ap))
```

Average Precision = 0.8665

实测结果（来自 [proj5.ipynb](#) 运行输出）：

- 初始分类器（训练后直接检测）平均精度（AP） = 0.8476
- 加入难负样本并重新训练后的平均精度（AP） = 0.8665

设计决策与讨论

- 为什么在 HoG 空间先计算整图的 HoG：避免对每个滑动窗口重复计算梯度与细胞块，速度更快。
- 镜像增强：增加正样本的多样性，常用于人脸/头部检测。
- 难负样本挖掘：用当前分类器挖掘假阳性并加入训练集可以提升分类器对“易混淆”背景的鲁棒性。对于简单的线性模板，在正负样本充足时提升有限，但仍是标准流程。
- 多尺度策略与 `scale_factor` 的选择：较小的缩放因子（更多尺度）有助于覆盖不同大小目标，但会显著增加计算量；0.85 在精度与速度间取得折中。